

Übung 0

17. März 2026

Aufgabe 01: Schachbrettmuster.....

Implementieren Sie eine öffentliche Klassenmethode `createChessBoard` in der Klasse `Task01`, die ein Schachbrettmuster mit den **Buchstaben** `X` und `0` im Wechsel erzeugt und als **String** zurückgibt. Die Klassenmethode hat folgenden Rückgabewert:

- **String**: Das erzeugte Schachbrettmuster.

Die Methode nimmt folgende Parameter entgegen:

- **row**: Anzahl der Zeilen.
- **col**: Anzahl der Spalten.

Die Methode soll die Parameter verwenden, um das Schachbrettmuster entsprechend zu generieren. Jede Zeile des Musters soll mit einem Zeilenumbruch abgeschlossen werden.

Falls ungültige Werte übergeben werden (z. B. `row <= 0` oder `col <= 0`), soll die Methode einen leeren String zurückgeben.

Für den Aufruf mit `row = 5` und `col = 5` wird folgender String erzeugt:

Erwartete Ausgabe für `row = 5` und `col = 5`

```
X 0 X 0 X
0 X 0 X 0
X 0 X 0 X
0 X 0 X 0
X 0 X 0 X
```

Hinweis

Verwenden Sie die bereitgestellte Datei `Task01Test.java`, um Ihre Implementierung zu überprüfen. Führen Sie die Unit Tests regelmäßig aus und prüfen Sie, ob alle Tests erfolgreich durchlaufen.

Aufgabe 02: Datenanalyse

In dieser Aufgabe implementieren Sie eine kleine Bibliothek zur Analyse numerischer Daten.

Die Analyse eines Datensatzes soll in einem eigenen Objekt gespeichert werden. Implementieren Sie daher zunächst eine Klasse **AnalysisResult**, die das Ergebnis der Datenanalyse kapselt.

(a) Erstellen Sie eine öffentliche Klasse **AnalysisResult**.

AnalysisResult
- min : double - max : double - sum : double - mean : double - sampleVariance : double
+ AnalysisResult(min : double, max : double, sum : double, mean : double, sampleVariance : double) + getMin() : double + getMax() : double + getSum() : double + getMean() : double + getSampleVariance() : double + toString() : String

Es sollen die folgenden Werte in den gekapselten Attributen gespeichert werden:

- **min**: Das Minimum der Werte.
- **max**: Das Maximum der Werte.
- **sum**: Die Summe aller Werte.
- **mean**: Der Mittelwert der Werte.
- **sampleVariance**: Die Stichprobenvarianz der Werte.

Implementieren Sie einen Konstruktor, der alle Werte entgegennimmt. Stellen Sie außerdem geeignete Getter-Methoden für alle Attribute bereit.

Implementieren Sie zusätzlich eine Methode

- **toString()**

die eine Textdarstellung des Analyseergebnisses zurückgibt.

Die Methode **toString()** soll eine Textdarstellung des Analyseergebnisses im folgenden Format zurückgeben:

```
AnalysisResult{min=1.0, max=10.0, sum=30.0, mean=6.0, sampleVariance=8.0}
```

Die Werte sollen in genau dieser Form und jeweils mit **einer Nachkommastelle** ausgegeben werden.

Hinweis

Verwenden Sie die bereitgestellte Datei **AnalysisResultTest.java**, um Ihre Implementierung zu überprüfen. Führen Sie die Unit Tests regelmäßig aus und prüfen Sie, ob alle Tests erfolgreich durchlaufen.

- (b) Implementieren Sie eine öffentliche Klasse **DataAnalyzer**. Diese Klasse soll Methoden bereitstellen, um ein Array vom Typ `double[]` zu analysieren.

DataAnalyzer
+ min(values : double[]) : double + max(values : double[]) : double + sum(values : double[]) : double + mean(values : double[]) : double + sampleVariance(values : double[]) : double + analyze(values : double[]) : AnalysisResult

Die Methode **analyze(...)** soll alle Kennwerte berechnen und ein Objekt der Klasse **AnalysisResult** zurückgeben. Die übrigen Methoden berechnen jeweils nur einen Wert. Diese Methoden können Sie anschließend als Hilfe verwenden.

Die Stichprobenvarianz ist definiert als

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

Dabei ist n die Anzahl der Werte im Array, x_i der i -te Wert im Array und $\bar{x}$ der Mittelwert aller Werte.

Hinweis

Verwenden Sie die bereitgestellte Datei **DataAnalyzerTest.java**, um Ihre Implementierung zu überprüfen. Führen Sie die Unit Tests regelmäßig aus und prüfen Sie, ob alle Tests erfolgreich durchlaufen.

- (c) Erweitern Sie die Klasse **DataAnalyzer** um folgende Methode ohne Rückgabewert:

– **sort(double[] values, boolean desc)**

Diese Methode soll das übergebene Array sortieren.

- Falls **desc == false**, soll das Array **aufsteigend** sortiert werden.
- Falls **desc == true**, soll das Array **absteigend** sortiert werden.

Implementieren Sie hierfür einen einfachen Sortieralgorithmus (z. B. **SelectionSort**).

Die Verwendung von Bibliotheksmethoden wie **Arrays.sort(...)** ist nicht erlaubt.

Hinweis

Verwenden Sie die bereitgestellte Datei **DataAnalyzerSortTest.java**, um Ihre Implementierung zu überprüfen. Führen Sie die Unit Tests regelmäßig aus und prüfen Sie, ob alle Tests erfolgreich durchlaufen.